Robotic Navigation and Exploration
HW2: Kinematic Models and Path Tracking Control
Due Sunday, March 29, 2026 (11:59 PM)

Student ID: M11452023 **Name:** 顏維廷

To complete this assignment, make sure you have the following tools installed on your computer:

1. Python version 3.8 +
2. opencv-python
3. numpy
4. matplotlib

Your written response must be typeset (LaTeX) and in blue. Also, do not forget to convert it into `report.pdf` before submission.

Please organize your submission into a folder as follows:

```
[your student ID]_HW2/
└ report.pdf
└ code/
    └ ...
```

Then, compress this folder into a zip archive named `[your student ID]_HW2.zip` and submit it to NTU COOL.
Warning: A 10-point penalty will be applied if your submission does not follow this format.

1 Introduction

This assignment focuses on the implementation and simulation of three fundamental **wheeled vehicle kinematic models**, alongside various **path-tracking control algorithms**. The primary objective is to enable an autonomous robot to robustly track a reference trajectory (generated in HW1) within a simulated environment.

The control architecture is decoupled into two parallel systems: **Longitudinal Control** and **Lateral Control**. First, you will build kinematic models for Basic, Differential Drive, and Bicycle configurations. Subsequently, longitudinal control algorithms will be implemented to generate safe velocity profiles and regulate the vehicle's speed along the track. Finally, you will develop multiple lateral tracking controllers—including PID, Pure Pursuit, Stanley, and LQR—to precisely minimize cross-track and heading errors. The assignment concludes by deploying these algorithms on simulated F1 circuits to evaluate their performance.

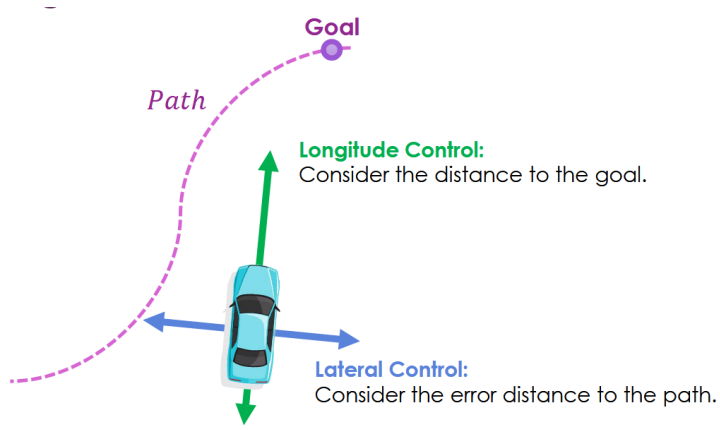


Figure 1: Longitudinal Control and Lateral Control

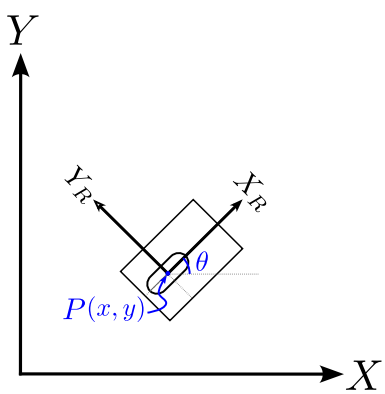
2 Kinematic Models

To simulate realistic vehicle motion, this section formulates the state-update logic for three distinct kinematic models. Each mathematical model dictates how the vehicle's **state** $\boldsymbol{\zeta} = [x, y, \theta]^T$ evolves continuously subject to a given **control input** $\boldsymbol{u}$.

In the provided codebase, the `State` object encapsulates not only the pose (x, y, θ) but also the vehicle's speed v and angular velocity ω. This formulation is particularly advantageous for models such as the bicycle model, where the direct control inputs are longitudinal acceleration a and steering angle δ; maintaining these dynamic states simplifies the discrete-time kinematic update. Note that throughout this document, (x, y) denotes the 2D position, and θ represents the global heading (orientation).

2.1 Basic Model

The basic model assumes that the robot can be approximated as a single wheel rolling on a plane. It can move forward or rotate but cannot move sideways.



- Control State $\boldsymbol{u} = [v, \omega]^T$, where v is the linear velocity and ω is the angular velocity.
- State: $\boldsymbol{\zeta} = [x, y, \theta]^T$, where x, y is the position and θ is the orientation of the vehicle.

$$\dot{\boldsymbol{\zeta}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = R(\theta) \begin{bmatrix} \dot{x_R} \\ \dot{y_R} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ 0 \\ \omega \end{bmatrix} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ \omega \end{bmatrix} \tag{1}$$

1. Check the code in `Simulation/kinematic_basic.py`, and answer the following questions.
   (a) (2 pts) What is the unit of `yaw`?
   degree
   (b) (2 pts) Why do we need `yaw = (state.yaw + state.w * self.dt) % 360`? What would happen without this line?

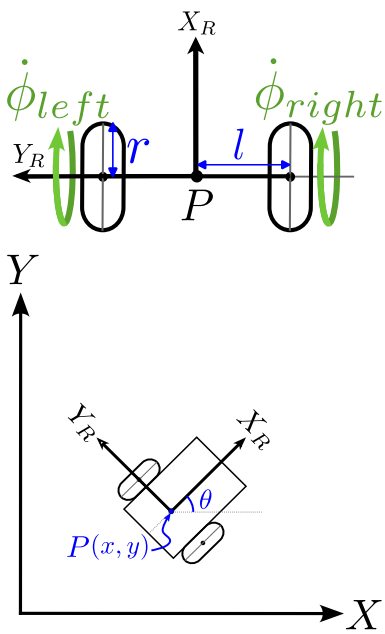
It keeps the angle between 0 and 360. Without it, the angle would keep growing, causing errors in path-tracking math

2. (2 pts) In practice, a controller may output infeasible commands (e.g., exceeding the actuator limits). Therefore, our simulator applies constraints to the current state and/or control inputs. Please check the code in `Simulation/simulator_basic.py` and list the constraints that are enforced.

   - Velocity limit: Speed is capped between a minimum and maximum value.
   - Angular velocity limit: Rotation speed is restricted to the actuator's range.

2.2 Differential Drive Model

The differential drive model features two independently driven wheels on a shared axis. In this model, the control input becomes $[\dot{\phi}_{\text{left}}, \dot{\phi}_{\text{right}}]^T$, representing the angular velocities of the left and right wheels, respectively.



- Control Input $\boldsymbol{u} = [\dot{\phi}_{\text{left}}, \dot{\phi}_{\text{right}}]^T$ The control input can be mapped to $[v, \omega]$ by the following formula:

$$v = \frac{r}{2}(\dot{\phi}_{\text{left}} + \dot{\phi}_{\text{right}})$$
$$\omega = \frac{r}{2l}(\dot{\phi}_{\text{right}} - \dot{\phi}_{\text{left}})$$

- State: $\boldsymbol{\zeta} = [x, y, \theta]^T$, where x, y is the position and θ is the orientation of the vehicle.

$$\dot{\boldsymbol{\zeta}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ \omega \end{bmatrix} = \begin{bmatrix} \frac{r}{2}(\dot{\phi}_{\text{left}} + \dot{\phi}_{\text{right}})\cos\theta \\ \frac{r}{2}(\dot{\phi}_{\text{left}} + \dot{\phi}_{\text{right}})\sin\theta \\ \frac{r}{2l}(\dot{\phi}_{\text{right}} - \dot{\phi}_{\text{left}}) \end{bmatrix} \tag{2}$$

Related code in `Simulation/kinematic_different_drive.py`

For more details and derivations, please refer to the lecture slides (pp.24).

1. (2 pts) Similar to 2.1.2, check the code in `Simulation/simulator_different_drive.py` and list the constraints that are enforced.

   - Motor Speed Limit: The angular velocities of the left and right wheels are restricted by the physical limits of the motors.
   - Actuator Range: Control inputs must stay within the minimum and maximum values defined in the simulator settings.

2. (4 pts) Complete `## TODO 2.2.2` in `navigation.py`. For the following lateral control algorithm, the basic and differential drive models are implemented in the same file. However, for the differential drive model, you need to convert the control input $[v, \omega]$ into $[\dot{\phi}_{\text{left}}, \dot{\phi}_{\text{right}}]$. Run the command below to check the result of differential drive model:

```
$ python navigation.py -s diff_drive -c pid -t 400mRunningTrack
```

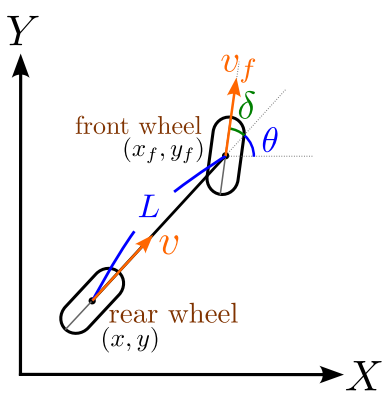
you should have similar result by running the following command:

```
$ python navigation.py -s basic -c pid -t 400mRunningTrack
```

```
        # Retrieve physical parameters from the simulator
        r = simulator.r
        l = simulator.l

        # Adding/Subtracting these equations gives us the individual wheel speeds
        next_lw = (2 * next_v - next_w * l) / (2 * r)
        next_rw = (2 * next_v + next_w * l) / (2 * r)
```

2.3 Bicycle Model

The bicycle model is a simplified mathematical representation of a four-wheeled vehicle's dynamics, reducing it to two wheels (one front, one rear) on a single track.



- Control Input $\boldsymbol{u} = [a, \delta]^T$, where a is the linear acceleration and δ is the steering angle.
- State: $\boldsymbol{\zeta} = [x, y, \theta]^T$, where x, y is the position and θ is the orientation of the vehicle.

$$\dot{\boldsymbol{\zeta}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ \omega \end{bmatrix} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ \frac{v}{L}\tan\delta \end{bmatrix} \tag{3}$$

where v is the velocity, L is the wheelbase, and δ is the steering angle.

For more details and derivations, please refer to the lecture slides (pp.32).

1. (5 pts) According to the above formula, complete the `step()` function in the `KinematicModelBicycle` class which is defined in the file `Simulation/kinematic_bicycle.py`

```
$ python navigation.py -s bicycle -c pid
```

```
        v = state.v + cstate.a * self.dt

        # 2. Convert degrees to radians for trigonometric calculations
        yaw_rad = np.deg2rad(state.yaw)
        delta_rad = np.deg2rad(cstate.delta)

        # 3. Calculate derivatives based on kinematic equations
        dx = v * np.cos(yaw_rad)
        dy = v * np.sin(yaw_rad)
        dyaw_rad = (v / self.l) * np.tan(delta_rad)

        # 4. Integrate to find the next state (Euler integration)
        x = state.x + dx * self.dt
        y = state.y + dy * self.dt
```

```
        # Convert angular change back to degrees for the State object
        yaw = state.yaw + np.rad2deg(dyaw_rad) * self.dt

        # 5. Set angular velocity (w) in degrees per second
        w = np.rad2deg(dyaw_rad)
```

2. (3 pts) Similar to 2.1.2, check the code in `Simulation/simulator_bicycle.py` and list the constraints that are enforced.

   - Acceleration limit: How fast the car can speed up or slow down.
   - Steering angle limit: The maximum angle the front wheel can turn.

3 Path Tracking - Longitudinal Control

In Homework 1, you generated the path geometry (waypoints). However, a raw path isn't enough for tracking; the vehicle needs a velocity profile to know when to slow down for corners and stay stable.

1. Following the methodology below, complete the `generate_speed_profile()` function in `trajectory_generator.py`

   Using these (x, y) waypoints, compute a reference velocity profile v_{ref} for the entire trajectory. The velocity profile should be both **safe** (limited by curvature) and **dynamically feasible** (smooth acceleration/deceleration).

   Algorithm outline:

   (a) **Curvature κ:** Compute the curvature at each waypoint from the derivatives of the path coordinates. (already written)

   (b) **Speed limit (4 pts):** Compute the maximum allowable speed based on lateral acceleration ($a_{\text{lat}} = 30\text{m/s}^2$) and a global speed cap ($v_{\max} = 85\text{m/s}$):

   $$v_{\text{limit}} = \min\left(v_{\max}, \sqrt{\frac{a_{\text{lat}}}{|\kappa|}}\right).$$

   Hint: In practice, κ can be zero on straight segments. To avoid division by zero, add a small constant (e.g., $\epsilon = 10^{-6}$) to the denominator: $|\kappa| + \epsilon$.

   (c) **Longitudinal smoothing (8 pts):** Apply a forward–backward pass ensuring the velocity profile respects acceleration and braking limits.

   - **Forward pass:** limit acceleration ($a_{\text{acc}} = 12\text{m/s}^2$).

   $$v_i^2 \leq v_{i-1}^2 + 2 \cdot a_{\text{acc}} \cdot ds$$

   - **Backward pass:** limit deceleration ($a_{\text{dec}} = 18\text{m/s}^2$) so the vehicle slows down before corners.

   $$v_i^2 \leq v_{i+1}^2 + 2 \cdot a_{\text{dec}} \cdot ds$$

   - **Launch Restrictions:**
     - Starting Speed ($v_{\text{ref}}[0]$): 5.0 m/s
     - Terminal Target Speed ($v_{\text{ref}}[-1]$): 0.0 m/s

Hint: Think carefully about where to apply each restriction.

Run the following command to check your velocity profile:

```
$ python trajectory_generator.py
```

```
#Speed limit from curvature
v_ref = np.sqrt(max_lat_acc / (curvature + 1e-6))
v_ref = np.clip(v_ref, 0, max_v)

#Longitudinal Smoothing
v_ref[0] = 5.0 # Start speed
v_ref[-1] = 0.0 # End speed

dx = np.diff(path_x)
dy = np.diff(path_y)
ds_seg = np.sqrt(dx**2 + dy**2)

# 1. Forward Pass: Limit acceleration
for i in range(1, len(v_ref)):
    v_max_forward = np.sqrt(v_ref[i-1]**2 + 2.0 * max_long_acc * ds_seg[i
        -1])
    if v_ref[i] > v_max_forward:
        v_ref[i] = v_max_forward

# 2. Backward Pass: Limit deceleration
for i in range(len(v_ref) - 2, -1, -1):
    v_max_backward = np.sqrt(v_ref[i+1]**2 + 2.0 * max_long_dec * ds_seg[i])
    if v_ref[i] > v_max_backward:
        v_ref[i] = v_max_backward
```

2. (8 pts) **Speed Tracking with PID:** In our bicycle model, the longitudinal control input is the vehicle's acceleration a. Please implement a PID controller that calculates the appropriate acceleration to effectively track the reference speed v_{ref}. The default gains K_p, K_i, and K_d are already provided in the code, but feel free to fine-tune them!

PID formula:

$$e(t) = v_{\text{ref}}(t) - v(t), \qquad u(t) = a = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}.$$

Run the following command to check your longitudinal PID controller:

```
$ python navigation.py -s bicycle -c pid -t 1000mStraight
```

```
# 1. Calculate the velocity error (e = v_ref - v)
error = v_ref - v

# 2. Update the accumulated error (integral term)
self.acc_ep += error * self.dt

# 3. Calculate the change in error (derivative term)
diff_ep = (error - self.last_ep) / self.dt
```

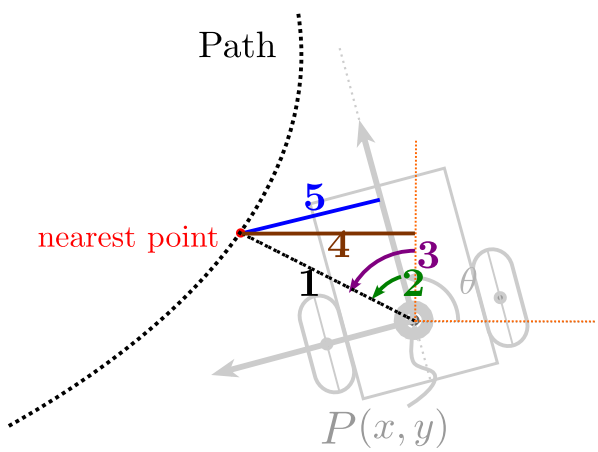
```
        # 4. Compute PID output: acceleration a = Kp*e + Ki*integral + Kd*
            derivative
        next_a = self.kp * error + self.ki * self.acc_ep + self.kd * diff_ep

        # 5. Store current error for the next time step
        self.last_ep = error

        # 6. Constrain the acceleration within the vehicle's physical limits (
            a_range)
        next_a = np.clip(next_a, self.a_range[0], self.a_range[1])
```

4 Path Tracking - Lateral Control

4.1 PID Control

PID control works just as well for lateral tracking. The main difference is simply how we define the tracking error.

1. (2 pts) According to `PathTracking/controller_pid_basic.py`, identify the error term of PID lateral control.



1

2. (5 pts) Tune the PID gains K_p, K_i, and K_d in `PathTracking/controller_pid_basic.py` to achieve the best performance, and briefly describe how you tuned the PID gains K_p, K_i, and K_d. It is recommended to include a screenshot of the resulting trajectory.
Hint: You may refer to PID Tuning Guide (or any other PID-tuning method).

```
$ python navigation.py -s basic -c pid -t 1000mStraight --init_shift 1.5
```

First, I set K_p to 0.5 to follow the path gently. Then, I increased K_d to 1.0 to stop the shaking. I kept K_i at 0.0 to prevent extra oscillations.

```
        def __init__(self, model,
            # TODO 4.1.2: Tune PID Gains
            kp=0.5, #1.0 -> 0.5
            ki=0.0,
            kd=1.0): #0.0 -> 1.0
```

3. (5 pts) For the bicycle model, you may choose whether to use the same error definition as the basic model. If not, please describe the error you used. Complete `PathTracking/controller_pid_bicycle.py`
Run the following command to check your PID controller:

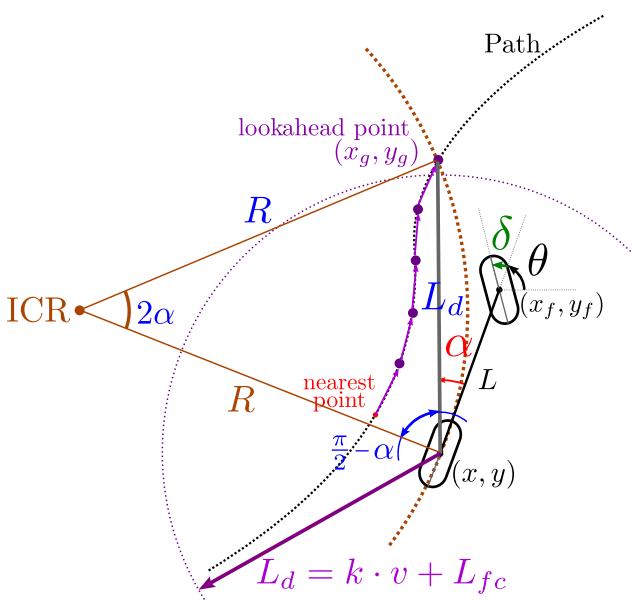
```
$ python navigation.py -s bicycle -c pid -t 400mRunningTrack
```

```
        # 1. Get the target point from the path
        target = self.path[self.current_idx]

        # 2. Calculate Cross-Track Error (CTE)
        dx = target[0] - x
        dy = target[1] - y
        error = dy * np.cos(np.deg2rad(yaw)) - dx * np.sin(np.deg2rad(yaw))

        # 3. PID calculation
        self.acc_ep += error * self.dt
        diff_ep = (error - self.last_ep) / self.dt
        next_delta = self.kp * error + self.ki * self.acc_ep + self.kd * diff_ep
        self.last_ep = error
```

your answer (if you changed the error definition)

4.2 Pure Pursuit Control

The Pure Pursuit algorithm calculates the steering angle δ needed to guide the vehicle toward a "look-ahead" point on the path.



Pure Pursuit Algorithm

1. Find the nearest point on the path.
2. Find the first point on the path that is at least L_d away.

$$L_d = k \times v + L_{fc}$$

3. Calculate the steering angle δ to move the vehicle toward the "look-ahead" point.

$$\delta = \arctan\left(\frac{2L\sin(\alpha)}{L_d}\right)$$

For more details and derivations, please refer to the lecture slides (pp. 34).

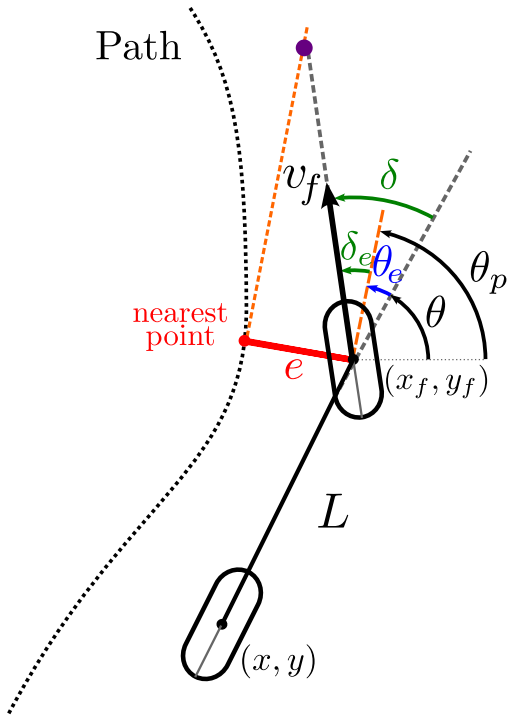
1. (8 pts) According to the above algorithm, complete `PathTracking/controller_pure_pursuit_bicycle.py`

```
$ python navigation.py -s bicycle -c pure_pursuit -t 400mRunningTrack
```

```
        # 1. Find the lookahead point
        lookahead_idx = self.current_idx
        for i in range(self.current_idx, len(self.path)):
            tx, ty = self.path[i][0], self.path[i][1]
            distance = np.sqrt((tx - x)**2 + (ty - y)**2)
            if distance >= Ld:
                lookahead_idx = i
                break

        self.current_idx = lookahead_idx
        target = self.path[self.current_idx]
        tx, ty = target[0], target[1]

        # 2. Calculate the angle to the target relative to the world frame
        target_angle_world = np.arctan2(ty - y, tx - x)

        # 3. Calculate Alpha
        current_yaw_rad = np.deg2rad(yaw)
        alpha = utils.angle_norm(target_angle_world - current_yaw_rad)

        # 4. Calculate Steering Angle (delta)
        delta_rad = np.arctan2(2.0 * self.l * np.sin(alpha), Ld)

        # 5. Convert to degrees
        next_delta = np.rad2deg(delta_rad)
```

4.3 Stanley Control

Stanley control is a path-tracking algorithm that determines the steering angle by combining the **heading error** relative to the path with the **cross-track error** measured from the center of the front axle to the nearest point on the path.



Stanley Control Algorithm

1. Find the nearest point on the path.
2. Calculate the heading error θ_e and the cross-track error δ_e.

$$\theta_e = \theta_{path} - \theta$$

$$|e| = \sqrt{(x - x_{path})^2 + (y - y_{path})^2}$$

3. Calculate the steering angle δ to move the vehicle toward the "look-ahead" point.

$$\delta = \theta_e + \delta_e = \theta_e + \arctan\left(\frac{-ke}{v_f}\right) \tag{4}$$

For more details and derivations, please refer to the lecture slides (pp. 35).

1. (8 pts) According to the above algorithm, complete `PathTracking/controller_stanley_bicycle.py`

```
$ python navigation.py -s bicycle -c stanley -t 400mRunningTrack
```

Hint1: Pay attention to the magnitude of θ_e
Hint2: Be careful about the sign of e, where $\dot{e} = v_f \sin(\delta_e)$ in deviation of Eq. (4).

```
# 1. Heading error
target_yaw = np.deg2rad(target[2])
current_yaw = np.deg2rad(yaw)
theta_e = utils.angle_norm(target_yaw - current_yaw)

while theta_e > np.pi:
    theta_e -= 2 * np.pi
while theta_e < -np.pi:
    theta_e += 2 * np.pi

# 2. Cross-track error (e)
error = np.sin(target_yaw) * (front_x - target[0]) - np.cos(target_yaw) * (
    front_y - target[1])

# 3. Stanley formula
ks = 5.0
delta_e = np.arctan2(self.kp * error, vf + ks)
next_delta = np.rad2deg(theta_e + delta_e)
next_delta = np.clip(next_delta, -40.0, 40.0)
```

4.4 LQR Control

Problem Formulation

Given a discrete-time linear system:

$$\mathbf{x}_{k+1} = A\mathbf{x}_k + B\mathbf{u}_k \tag{5}$$

The goal of LQR is to find an optimal control sequence $\mathbf{u}_k$ that minimizes the overall cost function J:

$$J = \sum_{k=0}^{\infty} (\mathbf{x}_k^T Q \mathbf{x}_k + \mathbf{u}_k^T R \mathbf{u}_k) \tag{6}$$

- Q (State Weight Matrix): Penalizes deviations from the desired state (error).
- R (Control Weight Matrix): Penalizes the control effort (energy expenditure).

The Matrix P (Cost-to-Go)

The matrix P is the key to solving LQR and represents the "Cost-to-Go".

- **Value Function $V(\mathbf{x})$:** The value function $V(\mathbf{x}_k)$ tells us the minimum cost from our current state $\mathbf{x}_k$ all the way to infinity:

$$V(\mathbf{x}_k) = \min_{\mathbf{u}_k, \mathbf{u}_{k+1}, \ldots} \sum_{i=k}^{\infty} \left(\mathbf{x}_i^T Q \mathbf{x}_i + \mathbf{u}_i^T R \mathbf{u}_i\right) \tag{7}$$

  For LQR, this value function is quadratic and is defined by the matrix P:

$$V(\mathbf{x}_k) = \mathbf{x}_k^T P \mathbf{x}_k \tag{8}$$

  Then, we can write the recursive form of value function:

$$\begin{aligned} V(\mathbf{x}_k) &= \min_{\mathbf{u}} \{\mathbf{x}_k^T Q \mathbf{x}_k + \mathbf{u}_k^T R \mathbf{u}_k + V(x_{k+1})\} \\ &= \min_{\mathbf{u}} \{\mathbf{x}_k^T Q \mathbf{x}_k + \mathbf{u}_k^T R \mathbf{u}_k + \mathbf{x}_{k+1}^T P \mathbf{x}_{k+1}\} \end{aligned} \tag{9}$$

- **Significance:**
  - P condenses the entire future trajectory dynamics and costs into a single matrix.
  - It measures the total future cost starting from state $\mathbf{x}_k$.

To find P, we must solve the Discrete Algebraic Riccati Equation (DARE):

$$P = A^T P A - (A^T P B)(R + B^T P B)^{-1}(B^T P A) + Q \tag{10}$$

The related code can be found in the function `_solve_DARE()` in `PathTracking/controller_lqr_bicycle.py`.

The Optimal Control Law

Once P is obtained, the optimal control input $\mathbf{u}_k$ is a linear state feedback law:

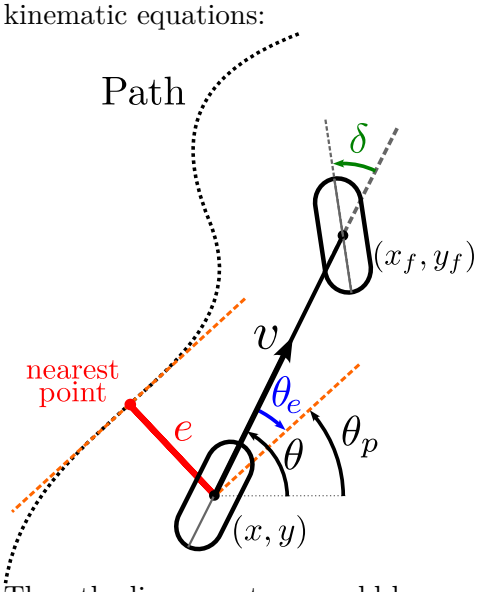
$$\mathbf{u}_k = -K\mathbf{x}_k \tag{11}$$

Where K is the optimal gain matrix derived from P:

$$K = (R + B^T PB)^{-1}(B^T PA) \tag{12}$$

The derivation process of K is detailed in lecture slides (pp.39).

1. (8 pts) We assume the state is $\mathbf{x} = [e, \theta_e]^T$, where e is the cross-track error and θ_e is the heading error, and the control input is $\mathbf{u} = [\delta]^T$. For the bicycle model, we have the following kinematic equations:



$$\dot{e} = -v\sin(\theta_e)$$
$$\dot{\theta}_e = -\frac{v}{L}\tan(\delta)$$

In LQR, we assume that θ_e and δ are small, so we can approximate $\sin(\theta_e) \approx \theta_e$ and $\tan(\delta) \approx \delta$, then we have

$$\dot{e} = -v\theta_e \tag{13}$$
$$\dot{\theta}_e = -\frac{v}{L}\delta \tag{14}$$

and

$$\begin{bmatrix} \dot{e} \\ \dot{\theta}_e \end{bmatrix} = \begin{bmatrix} 0 & -v \\ 0 & 0 \end{bmatrix} \begin{bmatrix} e \\ \theta_e \end{bmatrix} + \begin{bmatrix} 0 \\ -\frac{v}{L} \end{bmatrix} \delta$$

Then the linear system would be:

$$\begin{aligned} \mathbf{x}_{k+1} &= A\mathbf{x}_k + B\mathbf{u}_k \\ &= \begin{bmatrix} e_k \\ \theta_{e,k} \end{bmatrix} + \begin{bmatrix} \dot{e_k} \\ \dot{\theta}_{e,k} \end{bmatrix} \Delta t \\ &= \begin{bmatrix} 1 & -v \cdot \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} e_k \\ \theta_{e,k} \end{bmatrix} + \begin{bmatrix} 0 \\ -\frac{v}{L}\Delta t \end{bmatrix} \begin{bmatrix} \delta_k \end{bmatrix} \end{aligned}$$

Please refer to the equations above and complete the code in `PathTracking/controller_lqr_bicycle.py`

```
$ python navigation.py -s bicycle -c lqr -t 400mRunningTrack --lqr_control_state
    steering_angle
```

Hint1: Pay attention to the magnitude of θ_e
Hint2: Ensure that the definitions of θ_e, e, and δ in your code are consistent with Eq. (13) and (14).

```
        target_yaw = np.deg2rad(target[2])
        current_yaw = np.deg2rad(yaw)
```

```
        error = (y - target[1]) * np.cos(target_yaw) - (x - target[0]) * np.sin(
            target_yaw)
        theta_e = utils.angle_norm(current_yaw - target_yaw)
        while theta_e > np.pi:
            theta_e -= 2 * np.pi
        while theta_e < -np.pi:
            theta_e += 2 * np.pi

        state_vector = np.array([[error], [theta_e]])

        A = np.array([[1.0, v * self.dt],[0.0, 1.0]])
        B = np.array([[0.0],[v / self.l * self.dt]])

        P = self._solve_DARE(A, B, self.Q, self.R)
        K = np.linalg.inv(self.R + B.T @ P @ B) @ (B.T @ P @ A)

        u = -K @ state_vector
        next_delta = np.rad2deg(u[0, 0])
```

2. (4 pts) Write down how you choose the weight matrices Q and R
   I set a high value in Q for path error to stay on track, and a balanced R to keep steering smooth.

3. (4 pts) Now, to regulate the steering speed, we need to change the control input $\mathbf{u}$ to the steering angular velocity, $\dot{\delta}$. The state then becomes $\mathbf{x} = [e, \theta_e, \delta]^T$.
   Please finish the linear system below:

$$
\begin{aligned}
\mathbf{x}_{k+1} &= A\mathbf{x}_k + B\mathbf{u}_k \\
&= \begin{bmatrix} e_k \\ \theta_{e,k} \\ \delta_k \end{bmatrix} + \begin{bmatrix} \dot{e}_k \\ \dot{\theta}_{e,k} \\ \dot{\delta}_k \end{bmatrix} \Delta t \\
&= \begin{bmatrix} 1 & -v \cdot \Delta t & 0 \\ 0 & 1 & -\frac{v}{L} \cdot \Delta t \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} e_k \\ \theta_{e,k} \\ \delta_k \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \Delta t \end{bmatrix} \begin{bmatrix} \dot{\delta}_k \end{bmatrix}
\end{aligned}
$$

4. (6 pts) Based on the previous question, complete `PathTracking/controller_lqr_bicycle.py`

```
$ python navigation.py -s bicycle -c lqr -t 400mRunningTrack --lqr_control_state
    steering_angular_velocity
```

```
        # 1. Extract target and current values
        target_yaw = np.deg2rad(target[2])
        current_yaw = np.deg2rad(yaw)
        current_delta = np.deg2rad(delta)

        # 2. Calculate errors
        error = (y - target[1]) * np.cos(target_yaw) - (x - target[0]) * np.sin(
            target_yaw)
        theta_e = utils.angle_norm(current_yaw - target_yaw)
        while theta_e > np.pi:
            theta_e -= 2 * np.pi
```

```
        while theta_e < -np.pi:
            theta_e += 2 * np.pi

        # 3. State vector: [e, theta_e, delta]
        state_vector = np.array([
            [error],
            [theta_e],
            [current_delta]
        ])

         # 4. System matrices

        A = np.eye(3)
        A[0, 1] = v * self.dt
        A[1, 2] = v / self.l * self.dt

        B = np.zeros((3, 1))
        B[2, 0] = self.dt

        # 5. Solve DARE
        P = self._solve_DARE(A, B, self.Q, self.R)

        # 6. Compute gain K
        K = np.linalg.inv(self.R + B.T @ P @ B) @ (B.T @ P @ A)

        # 7. Control input
        u = -K @ state_vector
        delta_dot = u[0, 0]

        # 8. Integrate to get next steering angle
        next_delta = delta + np.rad2deg(delta_dot) * self.dt
        next_delta = np.clip(next_delta, -40.0, 40.0)
```

5 F1 Track Challenge

1. (10 pts) Now that you've built these models and controllers, it's time to put them to the test on some actual F1 circuits! We provide three F1 circuits: Silverstone, Suzuka, and Monza. You are free to choose whichever combination of kinematic model (basic, differential drive, or bicycle) and controller (PID, Pure Pursuit, Stanley, or LQR) that yields the best performance on average across all three tracks. You can tune controller parameters, implement additional controllers, or improve any part of the navigation flow to enhance overall system performance.

```
$ python navigation.py -s [basic|diff_drive|bicycle] -c [pid|pure_pursuit|stanley|
    lqr] -t [Silverstone|Suzuka|Monza]
```

Rules:

- Do not change the constraints $v_{\max}$, a_{lat}, a_{acc}, a_{dec} in velocity profiling or any simulator code.
- If you modify any code outside the designated `#TODO` blocks, please mark them with a `#NewFeature` comment and describe the changes in your report.

- Any violation of the above rules will receive 0 points in this challenge!

Grading Policy: Let $T_{\text{ref},i}$ be the baseline completion time for track i (**Silverstone: 107s, Suzuka: 115s, Monza: 94s**).

$$
\begin{aligned}
S_{T,i} &= \max\left(0,\ \min\left(6,\ 4 - k_T(T_i - T_{\text{ref},i})\right)\right) \\
S_{E,i} &= \max\left(0,\ \min\left(6,\ 4 - k_E(E_i - 1.5)\right)\right) \\
\text{Score}_i &= \min\left(10,\ S_{T,i} + S_{E,i}\right) \\
\text{Final Score} &= \left\lceil \frac{1}{3} \sum_i \text{Score}_i \right\rceil
\end{aligned}
$$

Here, $k_T = 0.5$ and $k_E = 2.0$ are penalty coefficients. Note that tracking error is penalized more heavily than completion time, so prioritize accuracy over speed. Try your best!

Write down your command for this challenge, and put the screenshot of your result.

```
$ python navigation.py -s bicycle -c lqr -t [Silverstone|Suzuka|Monza]
```

(Screenshot)

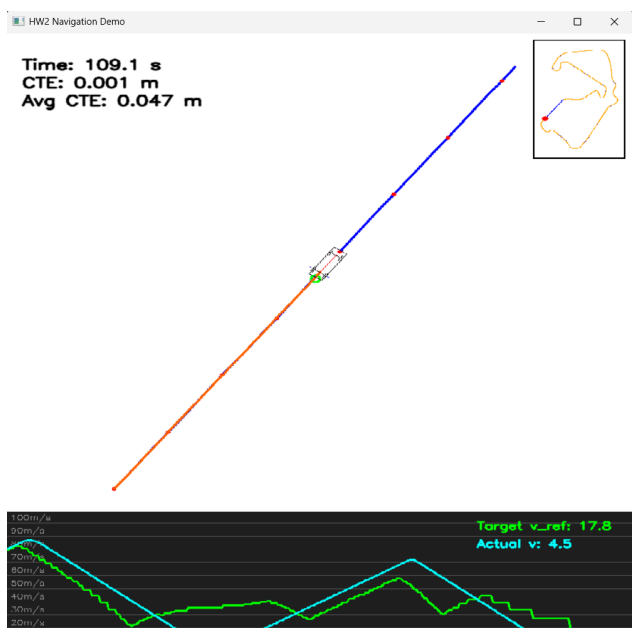


Figure 2: LQR Bicycle on Silverstone (steering angle)

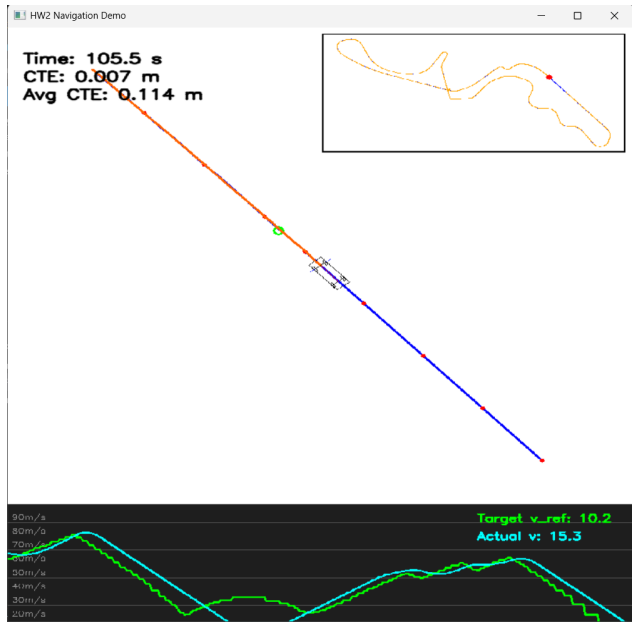


Figure 3: LQR Bicycle on Suzuka (steering angle)

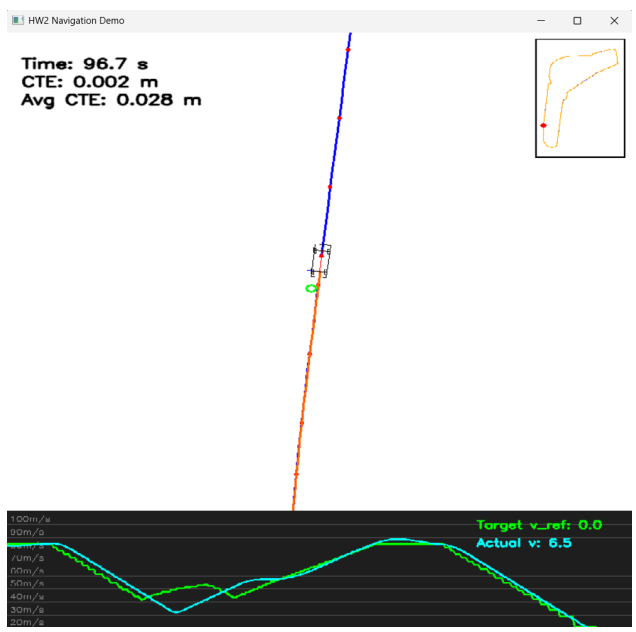


Figure 4: LQR Bicycle on Monza (steering angle)

```
        if control_state == 'steering_angle':
            self.Q = np.eye(2)
            self.R = np.eye(1)
            # TODO 4.4.1: Tune LQR Gains
            self.Q[0,0] = 1
            self.Q[1,1] = 10
            self.R[0,0] = 1
        elif control_state == 'steering_angular_velocity':
```

```
        self.Q = np.eye(3)
        self.R = np.eye(1)
        # TODO 4.4.4: Tune LQR Gains
        self.Q[0,0] = 1
        self.Q[1,1] = 10
        self.Q[2,2] = 100
        self.R[0,0] = 1
```

2. (Extra up to +5 pts) Describe what you have tried to improve the performance:
   I implemented a local path search strategy by limiting the target search range to a small window around the current index. This prevents the controller from incorrectly jumping to nearby or overlapping track segments at intersections, ensuring the vehicle stays on the correct path

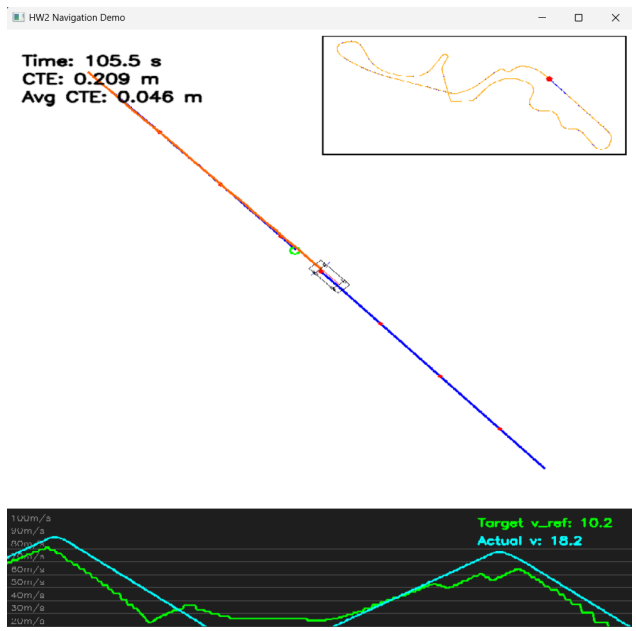


Figure 5: After: LQR Bicycle on Suzuka (steering angle)

6 Feedback (Optional)

1. (Extra up to +5 pts) We would love to hear your feedback on this homework! Please write down your thoughts, any difficulties you encountered, or suggestions for improving this assignment. Your feedback will be highly valuable for future improvements.

   `PathTracking/controller_pure_pursuit_bicycle.py` 中的 TODO 代號好像寫錯了，應該是 4.2.1

```
# TODO 4.3.1: Pure Pursuit Control for Bicycle Kinematic Model
```

Assignment Checklist

Please check the boxes (☐ → ■) which you have completed

Done	No.	Type	Task	Points
■	2.1.1	✎	Answer questions (yaw unit, update) about `kinematic_basic.py`	4
■	2.1.2	✎	List constraints about `simulator_basic.py`	2
■	2.2.1	✎	List constraints regarding `simulator_different_drive.py`	2
■	2.2.2	🐍	Complete # TODO 2.2.2 in `navigation.py` for differential drive	4
■	2.3.1	🐍	Complete `step()` in `KinematicModelBicycle`	5
■	2.3.2	✎	List constraints regarding `simulator_bicycle.py`	3
■	3.1	🐍	Complete velocity profiling in `trajectory_generator.py` (Speed limit 4 pts + Smoothing 8 pts)	12
■	3.2	🐍	Implement Longitudinal PID control in `navigation.py`	8
■	4.1.1	✎	Identify error term in `PathTracking/controller_pid_basic.py`	2
■	4.1.2	🐍/✎	Tune PID gains in `controller_pid_basic.py` and describe tuning process	5
■	4.1.3	🐍/✎	Complete `PathTracking/controller_pid_bicycle.py`	5
■	4.2.1	🐍	Complete `PathTracking/controller_pure_pursuit_bicycle.py`	8
■	4.3.1	🐍	Complete `PathTracking/controller_stanley_bicycle.py`	8
■	4.4.1	🐍	Complete `PathTracking/controller_lqr_bicycle.py`	8
■	4.4.2	✎	Write down how you choose the weight matrices Q and R	4
■	4.4.3	✎	Complete linear system of Equation (19)	4
■	4.4.4	🐍	Complete `PathTracking/controller_lqr_bicycle.py`	6
■	5.1	🐍/✎	F1 Track Challenge (Choose best model & controller)	10
■	5.2	✎	Describe attempts to improve performance (Extra)	+5
■	6.1	✎	(Optional) Provide feedback and suggestions	+5
			Total Score	**100** (+10)